

Graph Basics – Java Notes

Goal: Graph ko dekhkar turant samajhna ki `ArrayList` kaise banani hai aur Topological Sort DFS kaise lagana hai.

1. Graph ke Basic Terms

Graph mein mainly:

- **Vertex / Node** → graph ka point
- **Edge** → do nodes ke beech connection

Example:

```
0 → 1
```

Yahan:

```
0 = u  
1 = v  
  
Edge = [0, 1]
```

Matlab:

```
u → v
```

2. Graph Input ke Common Forms

Graph problems mein input mostly 3 forms mein milta hai:

1. Edge List

```
int[][] edges = {  
    {0, 1},  
    {0, 2},  
    {1, 3}  
};
```

2. Adjacency Matrix

```
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0
```

3. Adjacency List

```
0 → [1, 2]
1 → [3]
2 → [3]
3 → []
```

3. Adjacency List kya hoti hai?

Adjacency List mein har node ke saamne uske neighbours store hote hain.

Example:

```
0 → 1
0 → 2
1 → 3
2 → 3
```

Adjacency List:

```
0 → [1, 2]
1 → [3]
2 → [3]
3 → []
```

Java:

```
ArrayList<ArrayList<Integer>> adj = new ArrayList<>();
```

4. Empty Adjacency List kaise banaye?

Agar vertices hain:

```
ArrayList<ArrayList<Integer>> adj = new ArrayList<>();

for (int i = 0; i < V; i++) {
    adj.add(new ArrayList<>());
}
```

Important

Agar:

V = 4

toh initially:

```
0 → []
1 → []
2 → []
3 → []
```

5. Edge List → Adjacency List

Suppose:

```
int[][] edges = {
    {0, 1},
    {0, 2},
    {1, 3},
    {2, 3}
};
```

Meaning:

```
0 → 1
0 → 2
1 → 3
2 → 3
```

Directed Graph

Pattern:

```
adj.get(u).add(v);
```

Complete code:

```
ArrayList<ArrayList<Integer>> adj = new ArrayList<>();

for (int i = 0; i < V; i++) {
    adj.add(new ArrayList<>());
}

for (int[] edge : edges) {

    int u = edge[0];
    int v = edge[1];

    adj.get(u).add(v);
}
```

Result:

```
0 → [1, 2]
1 → [3]
2 → [3]
3 → []
```

Remember

[u, v]

means

u → v

So:

```
adj.get(u).add(v);
```

One-line memory trick

Directed = u se v jao → `adj[u].add(v)`

6. Undirected Graph

Undirected graph mein:

```
0 -- 1
```

ka matlab hai:

```
0 → 1  
1 → 0
```

Isliye dono directions add karni hoti hain:

```
adj.get(u).add(v);  
adj.get(v).add(u);
```

Complete:

```
for (int[] edge : edges) {  
  
    int u = edge[0];  
    int v = edge[1];  
  
    adj.get(u).add(v);  
    adj.get(v).add(u);  
}
```

Result:

```
0 → [1]  
1 → [0]
```

Remember

Directed:

```
adj.get(u).add(v);
```

Undirected:

```
adj.get(u).add(v);  
adj.get(v).add(u);
```

Undirected = dono taraf connection

7. Adjacency Matrix → Adjacency List

Suppose matrix:

```
0 1 1 0  
0 0 0 1  
0 0 0 1  
0 0 0 0
```

Rule:

```
matrix[i][j] == 1
```

means:

```
i → j
```

Example:

```
matrix[0][1] = 1
```

means:

```
0 → 1
```

Java Conversion

```
ArrayList<ArrayList<Integer>> adj = new ArrayList<>();  
  
for (int i = 0; i < V; i++) {  
    adj.add(new ArrayList<>());  
}
```

```

for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        if (matrix[i][j] == 1) {
            adj.get(i).add(j);
        }
    }
}

```

Result:

```

0 → [1, 2]
1 → [3]
2 → [3]
3 → []

```



Matrix Shortcut

```

matrix[i][j] == 1
    ↓
    i → j
    ↓
adj.get(i).add(j)

```

8. Weighted Graph

Suppose:

```

0 → 1 (5)
0 → 2 (10)
1 → 3 (7)

```

Input:

```

int[][] edges = {
    {0, 1, 5},
    {0, 2, 10},
    {1, 3, 7}
};

```

Yahan:

```
edge[0] = u
edge[1] = v
edge[2] = weight
```

So:

```
int u = edge[0];
int v = edge[1];
int wt = edge[2];
```

Weighted graph mein neighbour ke saath weight bhi store karna padega.

9. Weighted Graph ka Pair

```
class Pair {

    int node;
    int weight;

    Pair(int node, int weight) {
        this.node = node;
        this.weight = weight;
    }
}
```

Adjacency List:

```
ArrayList<ArrayList<Pair>> adj = new ArrayList<>();
```

Conversion:

```
for (int i = 0; i < V; i++) {
    adj.add(new ArrayList<>());
}

for (int[] edge : edges) {

    int u = edge[0];
    int v = edge[1];
    int wt = edge[2];
```



```
adj.get(u).add(new Pair(v, wt));  
}
```

10. Graph Conversion ka Master Rule 🧠

Graph problem dekhte hi ye 3 questions pucho:

Question 1: Directed ya Undirected?

Directed

```
adj.get(u).add(v);
```

Undirected

```
adj.get(u).add(v);  
adj.get(v).add(u);
```

Question 2: Weighted ya Unweighted?

Unweighted

```
ArrayList<ArrayList<Integer>>
```

Weighted

```
ArrayList<ArrayList<Pair>>
```

Question 3: Input kis form mein hai?

Edges

```
for (int[] edge : edges)
```

Matrix

```
for (int i = 0; i < V; i++) {  
    for (int j = 0; j < V; j++) {
```

```
        if (matrix[i][j] == 1) {  
            ...  
        }  
    }  
}
```

11. Topological Sort

Topological Sort sirf:

Directed Acyclic Graph (DAG)

ke liye valid hota hai.

DAG:

Directed
+
No Cycle

Example:

0 → 1 → 3
↓
2 → 3

Possible Topological Order:

0 1 2 3

ya:

0 2 1 3

Dono correct ho sakte hain.

12. Topological Sort ki Definition

For every edge:

$u \rightarrow v$

Topological order mein:

u pehle
v baad mein

Example:

$0 \rightarrow 1$

Valid:

0 1

Invalid:

1 0

13. Topological Sort using DFS

DFS ka main idea:

1. Node par jao
2. Uske neighbours visit karo
3. Jab saare neighbours complete ho jaayein
4. Node ko Stack mein push karo
5. Finally stack ko pop karo

Golden Rule ★

DFS ke END mein node ko Stack mein push karo.

```
dfs(neighbour);  
  
stack.push(node);
```

14. Topological Sort – Complete Java Code

```
import java.util.*;

class Solution {

    void dfs(
        int node,
        ArrayList<ArrayList<Integer>> adj,
        boolean[] visited,
        Stack<Integer> stack
    ) {

        visited[node] = true;

        for (int neighbour : adj.get(node)) {

            if (!visited[neighbour]) {
                dfs(neighbour, adj, visited, stack);
            }
        }

        // DFS complete hone ke baad
        stack.push(node);
    }

    public ArrayList<Integer> topoSort(
        int V,
        int[][] edges
    ) {

        // 1. Create adjacency list
        ArrayList<ArrayList<Integer>> adj = new ArrayList<>();

        for (int i = 0; i < V; i++) {
            adj.add(new ArrayList<>());
        }

        // 2. Convert edges to adjacency list
        for (int[] edge : edges) {

            int u = edge[0];
            int v = edge[1];

            adj.get(u).add(v);
        }

        // 3. Visited array
        boolean[] visited = new boolean[V];
    }
}
```

```

// 4. Stack
Stack<Integer> stack = new Stack<>();

// 5. DFS
for (int i = 0; i < V; i++) {

    if (!visited[i]) {
        dfs(i, adj, visited, stack);
    }
}

// 6. Create answer
ArrayList<Integer> ans = new ArrayList<>();

while (!stack.isEmpty()) {
    ans.add(stack.pop());
}

return ans;
}
}

```

15. Topological Sort ka Dry Run

Graph:

```

0 → 1
0 → 2
1 → 3
2 → 3

```

Adjacency List:

```

0 → [1, 2]
1 → [3]
2 → [3]
3 → []

```

DFS:

```

DFS(0)
  ↓
DFS(1)
  ↓
DFS(3)

```

```
↓  
push(3)  
  
push(1)  
  
DFS(2)  
↓  
3 already visited  
  
push(2)  
  
push(0)
```

Stack:

```
[3, 1, 2, 0]
```

Pop:

```
0 2 1 3
```

Valid Topological Order:

```
0 2 1 3
```

16. `boolean[] visited` ka role

Ye confusion important hai.

```
boolean[] visited = new boolean[V];
```

Iska kaam hai:

Kya ye node already visit ho chuki hai?

Example:

```
visited[0] = true
```

means:

Node 0 visited hai.

Ye `boolean` answer return nahi kar raha.

Ye sirf tracking ke liye hai.

17. Topological Sort mein Stack kyun?

Suppose:

0 → 1

DFS:

```
DFS(0)
  ↓
DFS(1)
  ↓
push(1)
  ↓
push(0)
```

Stack mein:

[1, 0]

Pop:

0
1

Exactly:

0 → 1

Isliye:

DFS finish order ko reverse karne ke liye Stack use hota hai.

18. Complexity

Adjacency List banane mein:

$O(E)$

DFS:

$O(V + E)$

Overall:

Time = $O(V + E)$
Space = $O(V + E)$



FINAL REVISION CHEAT SHEET

Edge List → Directed

```
for (int[] edge : edges) {  
  
    int u = edge[0];  
    int v = edge[1];  
  
    adj.get(u).add(v);  
}
```

Edge List → Undirected

```
for (int[] edge : edges) {  
  
    int u = edge[0];  
    int v = edge[1];  
  
    adj.get(u).add(v);  
    adj.get(v).add(u);  
}
```


Matrix → Directed

```
for (int i = 0; i < V; i++) {  
    for (int j = 0; j < V; j++) {  
        if (matrix[i][j] == 1) {  
            adj.get(i).add(j);  
        }  
    }  
}
```

Weighted Graph

```
int u = edge[0];  
int v = edge[1];  
int wt = edge[2];  
  
adj.get(u).add(new Pair(v, wt));
```

Topological Sort DFS

```
visited[node] = true;  
  
for (int neighbour : adj.get(node)) {  
    if (!visited[neighbour]) {  
        dfs(neighbour, adj, visited, stack);  
    }  
}  
  
stack.push(node);
```

★ 5 Lines jo hamesha yaad rakhni hain

1. Directed

```
adj.get(u).add(v);
```

2. Undirected

```
adj.get(u).add(v);  
adj.get(v).add(u);
```

3. Matrix

```
if (matrix[i][j] == 1)
```

means:

```
i → j
```

4. Topological DFS

```
dfs(neighbour);  
stack.push(node);
```

5. Topological Sort

DFS karo → neighbours complete karo → node stack mein daalo → stack pop karo.

One-Minute Graph Thinking

Jab bhi graph question aaye:

```
INPUT  
↓  
Edges ya Matrix?  
↓  
Directed ya Undirected?  
↓  
Weighted ya Unweighted?  
↓  
Adjacency List banao  
↓  
DFS / BFS / Topological Sort / Dijkstra...
```

Bas pehle graph ko sahi representation mein convert karo.

Uske baad algorithm lagana kaafi easy ho jaata hai.